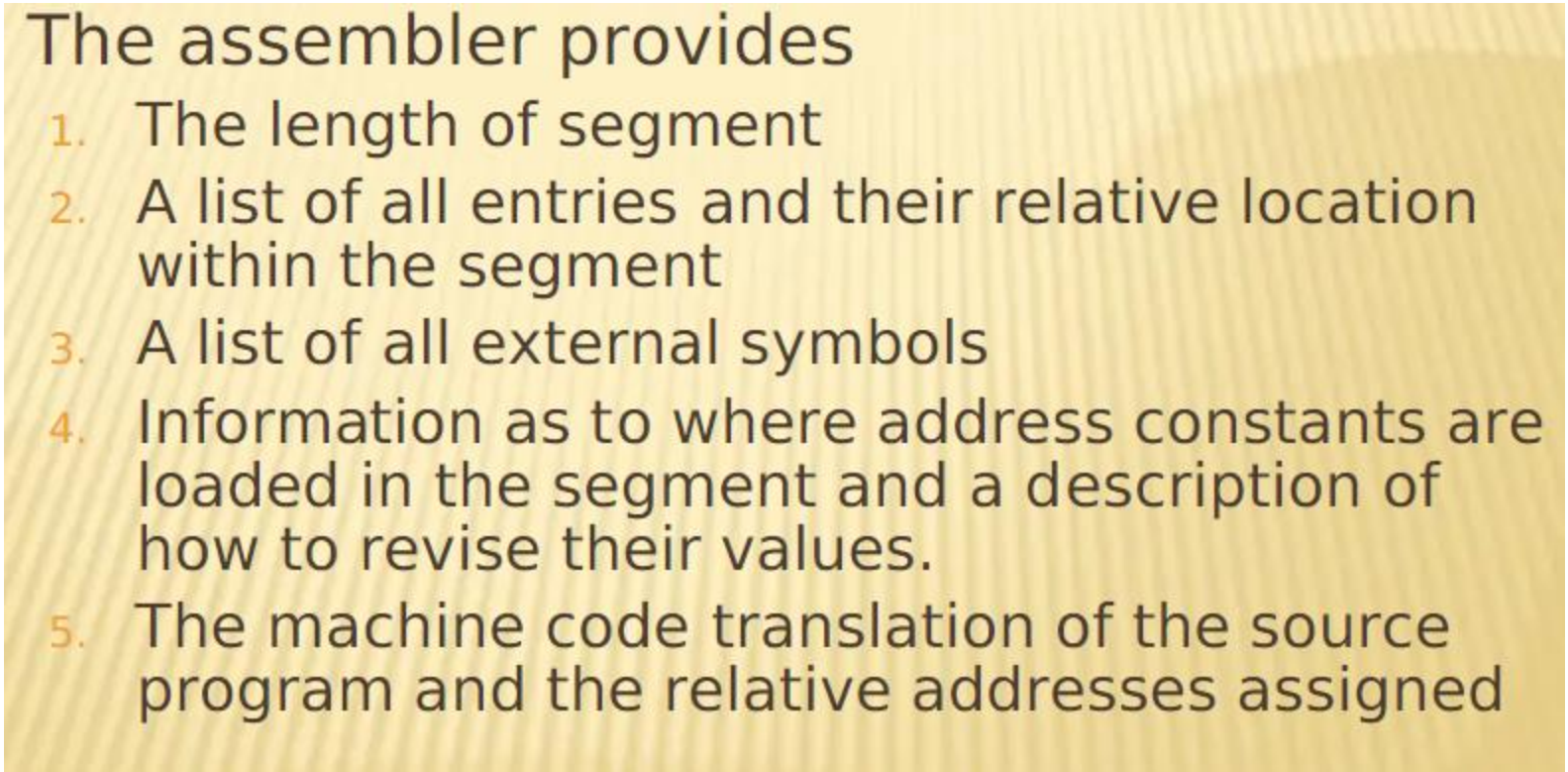
Unit 4

Direct Linking Loader

Direct Linking Loader

- A direct linking loader is a general relocatable loader, and is perhaps the most popular loading scheme presently used.
- The direct linking loader has the advantage of allowing the programmer multiple procedure segments and multiple data segments and of giving him complete freedom in referencing data or instructions contained in other segments.
- This provides flexibility intersegment referencing and accessing ability , while at the same time allowing independent translations of programs.

The assembler or translator must give the loader the following information with each procedure or data segment:



- There are four sections to the object deck (and four corresponding card formats):

1. External Symbol Dictionary card(ESD)
2. Instructions and data cards, called " text" of program (TXT)
3. Relocation and Linkage Dictionary card(RLD)
4. End card (END)

There are three types of external symbols, as illustrated in the above:

1. Segment Definition (SD)- name on START or CSECT card
2. Local Definition (LD)- specified on ENTRY card. There must be a label in same program with same name
3. External Reference (ER)- specified on EXTRN card. There must be a corresponding ENTRY, START, or CSECT card in another program with same name.

Each SD and ER symbol is assigned a unique number (e.g., 1,2,3…) by the assembler. This number is called the symbol's identifier, or ID and is used in conjunction with the RLD cards.

i. ***ESD card:*** It contains information about all symbols that are defined in the program but reference somewhere, It contains:

- Reference number
- Symbol name
- Type Id
- Relative location
- Length

There are again ESD cards classified into 3 types of mnemonics. They are:

i. SD [Segment Definition]: It refers to the segment definition
ii. LD; It refers to the local definition
iii. ER: it refers to the external reference they are used in the [EXTRN] pseudo op code

***ii. TXT Card*:** It contains the actual information are text which are already translated.

iii. RLD Card: This card contains information about location in the program whose contexts depends on the address at which the program is placed.

In this we are used '+' and '–'sign, when we are using the '+' sign then no need of relocation, when we are using '-'sign relocation is necessary.

The format of RLD contains:

i. Reference number
ii. Symbol
iii. Flag
iv. Length
v. Relative location

iv. END Card: It indicates end of the object program.

Program				Translation		
Card no.				Rel. loc.		
1.	JOHN	START				
2.		ENTRY	RESULT			
3.		EXTRN	SUM			
4.		BALR	12,0	0	BALR	12,0
5.		USING	*,12			
6.		ST	14,SAVE	2	ST	14,54(0,12)
7.		L	1,POINTER	6	L	1,46(0,12)
8.		L	15,ASUM	10	L	15,58(0,12)
9.		BALR	14,15	14	BALR	14,15
10		ST	1,RESULT	16	ST	1,50(0,12)
11.		L	14,SAVE	20	L	14,54(0,12)
12.		BR	14	24	BCR	15,14
				26	--	
13.	TABLE	DC	F'1,7,9,10,3'	28	1	
				32	7	
				36	9	
				40	10	
				44	3	
14.	POINTER	DC	A(TABLE)	48	28	
15.	RESULT	DS	F	52	--	
16.	SAVE	DS	F	56	--	
17.	ASUM	DC	A(SUM)	60	?	
18.		END		64		

ESD records

symbol	type	Rel-location	Length
JOHN	SD	0	64
RESULT	LD	52	
SUM	ER	—	—

RLD cards

Reference no.	*Symbol*	*Flag*	*Length*	*Relative location*
14	JOHN	+	4	48
17	SUM	+	4	60

TXT cards

Reference no.	Relative location	Object code	
4	0	BALR	12,0
6	2	ST	14,54(0,12)
7	6	L	1,46(0,12)
8	10	L	15,58(0,12)
9	14	BALR	14,15
10	16	ST	1,50(0,12)
11	20	L	14,54(0,12)
12	24	BCR	15,14
13	28	1	
13	32	7	
13	36	9	
13	40	10	
13	44	3	
13	48	28	
14	60	0	
17			

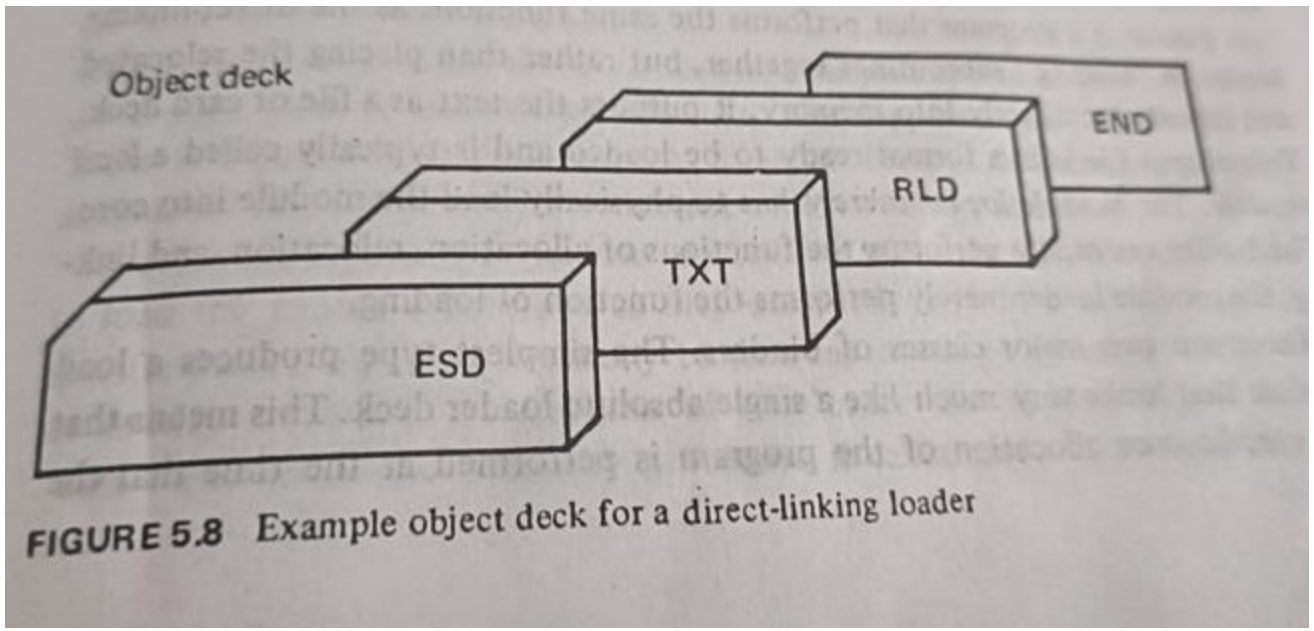


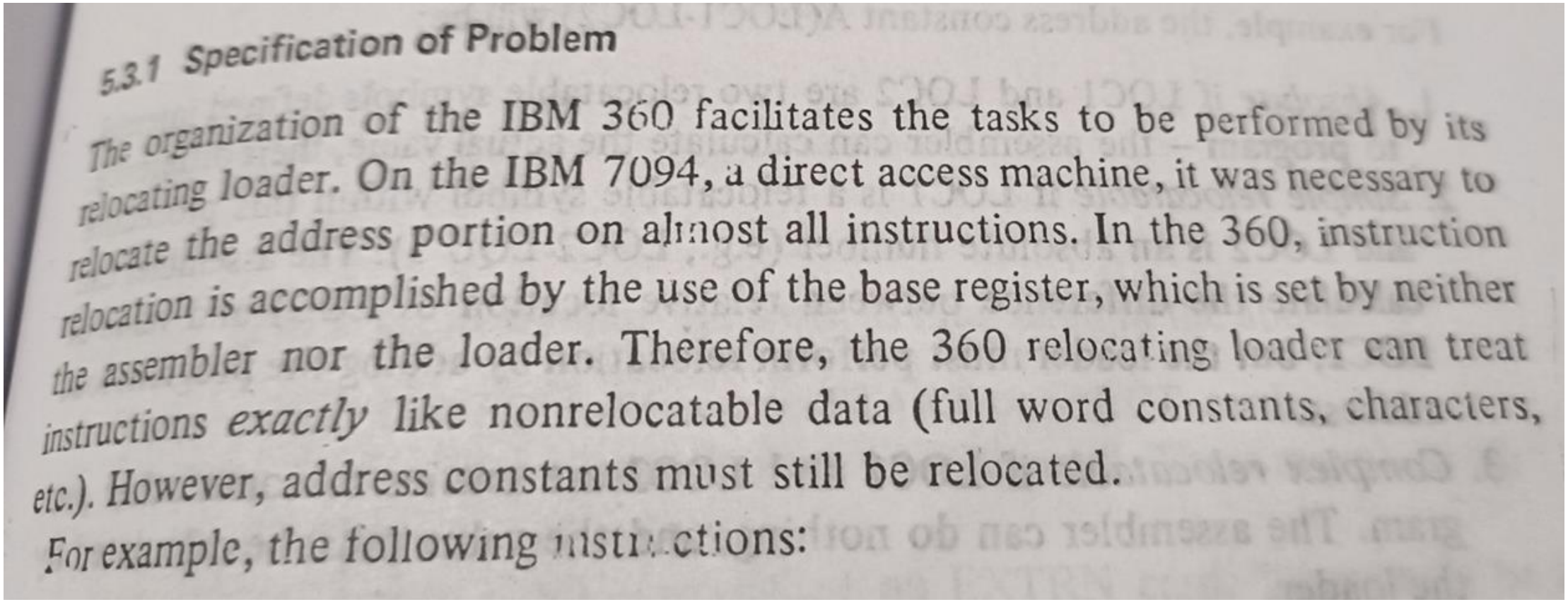
FIGURE 5.8 Example object deck for a direct-linking loader

Design of Direct linking loader

- In this section a design of an IBM 360 type direct linking loader is presented. Certain obscure features have been omitted, and where alternative formats are possible only the simplest is given.
- The design steps followed will parallel those taken in the design of an assembler. Note that because the direct linking loader needs to know the absolute values of external symbols before it can perform the modification on address constants, it requires two passes.
- Design steps

1. Specification of problem
2. Specification of data Structures
3. Format of Data Bases
4. Algorithm

1. Specification of Problem



5.3.1 Specification of Problem

The organization of the IBM 360 facilitates the tasks to be performed by its relocating loader. On the IBM 7094, a direct access machine, it was necessary to relocate the address portion on almost all instructions. In the 360, instruction relocation is accomplished by the use of the base register, which is set by neither the assembler nor the loader. Therefore, the 360 relocating loader can treat instructions *exactly* like nonrelocatable data (full word constants, characters, etc.). However, address constants must still be relocated.

For example, the following instructions:

```
TEST    START   0
        USING   *,15
        L       1,DATA
        .
        .
        .
DATA    DC      F'5'
        END
```

might be assembled as:

```
Rel. loc.   Instruction/data
    0       L   1,96(0,15).
    .           .
    .           .
    .           .
   96           5
```

Regardless of where the program is loaded in memory, as long as DATA remains 96 bytes from the beginning of the program, the contents of the base register (15) obviously will be different, depending upon the program's load location.

On the other hand, consider modifying the above example:

```
        .
        .
        .
DATA    DC      F'5'
DATALOC DC      A(DATA)
        END
```

DATALOC must contain the absolute address of DATA. The assembler knows only that DATA is 96 bytes from the beginning of the program, so the loader must add to this the load address of the program in finding the actual absolute address to be contained in DATALOC.

Let us clarify the scope of the address constant problem. An address constant may be (1) absolute; (2) simple relocatable; or (3) complex relocatable.

For example, the address constant A(LOC1-LOC2) will be:

1. *Absolute* if LOC1 and LOC2 are two relocatable symbols defined internal to program – the assembler can calculate the actual value, their difference.
2. *Simple relocatable* if LOC1 is a relocatable symbol within this procedure and LOC2 is an absolute number (e.g., LOC2 EQU 5). The assembler can calculate the difference between relative location of LOC1 and value of LOC2, but the loader must perform relocation by adding the program load address.
3. *Complex relocatable* if LOC1 and LOC2 are entries to some other program. The assembler can do nothing, and the value must be calculated by the loader.

- There are four sections to the object deck (and four corresponding card formats):

1. External Symbol Dictionary card(ESD)
2. Instructions and data cards, called " text" of program (TXT)
3. Relocation and Linkage Dictionary card(RLD)
4. End card (END)

There are three types of external symbols, as illustrated in the above:

1. Segment Definition (SD)- name on START or CSECT card
2. Local Definition (LD)- specified on ENTRY card. There must be a label in same program with same name
3. External Reference (ER)- specified on EXTRN card. There must be a corresponding ENTRY, START, or CSECT card in another program with same name.

Each SD and ER symbol is assigned a unique number (e.g., 1,2,3…) by the assembler. This number is called the symbol's identifier, or ID and is used in conjunction with the RLD cards.

Sample procedures PG1 and PG2

Source Card Reference	Relative Address		Sample Program(Source Deck)	
1	0 PG1	START		
2		ENTRY	PG1ENT1, PG1ENT2	
3		EXTRN	PG1ENT1, PG2	
4	20 PG1ENT1	_____		
5	30 PG1ENT2	_____		
6	40	DC	A(PG1ENT1)	
7	44	DC	A(PG1ENT2+15)	PG1
8	48	DC	A(PG2ENT2-PG1ENT1-3)	
9	52	DC	A(PG2)	
10	56	DC	A(PG2ENT1+PG2-PG1ENT1+4)	
11		END		
12	0 PG2	START		
13		ENTRY	PG2ENT1	
14		EXTRN	PG1ENT1,PG1ENT2	PG2
15	16 PG2ENT1	_____		
16	24	DC	A(PG1ENT1)	
17	28	DC	A(PG1ENT2+15)	
18	32	DC	A(PG1ENT2-PG1ENT1-3)	
19		END		

ESD Cards:
In ESD card table contains information necessary to build the external symbol dictionary or symbols table
In the above source code the symbols are PG1, PG1ENT2, PG2, and PG2ENT1

Format of ESD Card for PG1:

Source card Reference	Name	Type	Id	Relative address	length
1	PG1	SD	01	0	60
2	PG1ENT1	LD	-	20	-
2	PG1ENT2	LD	-	30	-
3	PG2	ER	-	-	-
3	PG2ENT1	ER	-	-	-

Explanation

Here, the PG1 is the segment definition it means, the header of program1.
PG1ENT1 and PG1ENT2 those are the local definition of program1, so that we are using the type LD.
PG2 and PG2ENT1 those are using the EXTRN pseudo op code, so that we are using the type ER.

Text card for PG1:

The format of card will be

Source card Reference	Relative address	Content	Comments
6	40-43	20	
7	44-47	45	=30+15
8	48-51	7	=30-20-3
9	52-55	0	Unknown to PG1
10	56-60	-16	-20+4

Text cards: The format of card will be

Source card reference	*Relative address*	*Content*	*Comments*
6	40-43	20	
7	44-47	45	=30+15
8	48-51	7	=30-20-3
9	52-55	0	Unknown to PG1
10	56-60	-16	-20+4

RLD Card:

Source card reference address	*ESD ID*	*Length [bytes]*	*Flag + or -*	*relative*
6	02	4	+	40
7	02	4	+	44
9	03	4	+	52
10	02	4	+	56
10	03	4	+	56
10	02	4	-	56

ESD Card for program2 [PG2]:

Source Card Reference	Name	Type	Id	ADDR	Length
12	PG2	SD	01	0	36
13	PG2ENT1	LD	-	16	-
14	PG1ENT1	ER	03	-	-
14	PG1ENT2	ER	03	-	-

Text Card for PG2:

Source card reference	Relative address	Content
16	24-27	0
17	28-31	15
18	32-35	-3

16=A (PG1ENT1) =0
17=A (PG1ENT2+15) =0+15=15
18=A (PG1ENT2-PG1ENT1-3) =0-0-3=-3

RLD Card for PG2:

Source card Reference	ESD ID	Length[flag] [bytes]	Flag + or -	Relative address
16	01	4	+	24
17	-	4	+	28
18	03	4	+	32
18	03	4	-	32

2. Specification of Data Structures

Specification of data structure: *Pass1 database:*

i. Input object decks
ii. *Initial Program Load Addresses [IPLA]:* The IPLA supplied by the programmer or operating system that specifies the address to load the first segment.
iii. *Program Load Address counter [PLA]:* It is used to keep track of each segments assigned location
iv. *Global External Symbol Table [GEST]:* It is used to store each external symbol and its corresponding assigned core address
v. A copy of the input to be used later by pass2
vi. A printed listing that specifies each external symbol and its assigned value

Pass2 database:

i. A copy of object program is input to pass2
ii. The Initial Program Load Address [IPLA]
iii. The Program Load Address counter [PLA]
iv. A table the Global External Symbol Table [GEST]
v. The Local External Symbol Array [LESA]: which is used to establish a correspondence between the ESD ID numbers used on ESD and RLD cards and the corresponding External symbols , Absolute address value

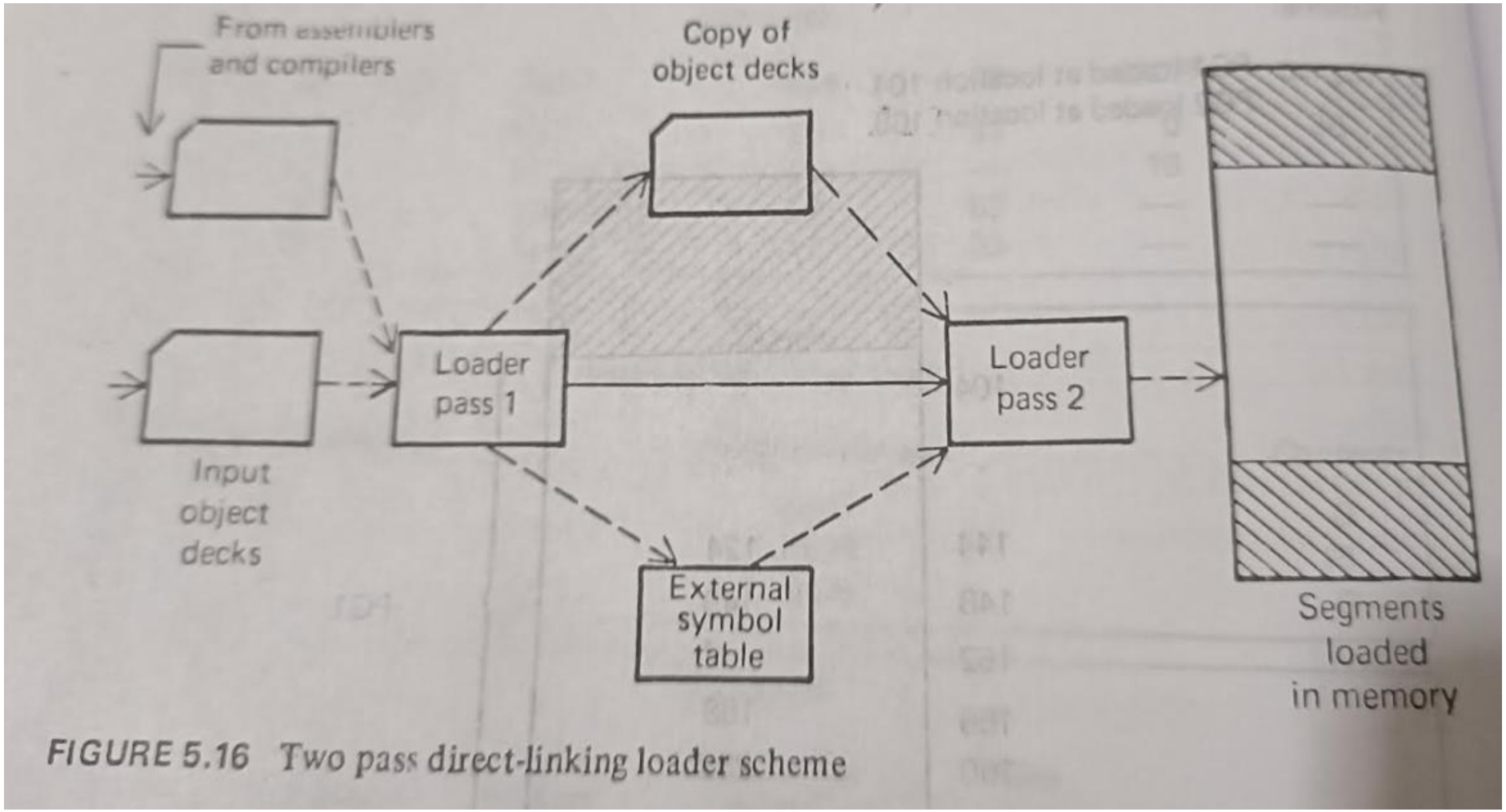


FIGURE 5.16 Two pass direct-linking loader scheme

3. Format of data bases

Format of data bases:

i. *Object deck:* The object deck contains 4 types of cards

ii. *ESD Card format:*

Source card reference	Name	Type	ID	Relative address	length

iii. *TEXT Card:*

Source card reference address	Relative address	content

iv. *RLD Card:*

Source card references	ESD ID	Length	Flag + or -	Relative address

v. *Global External Symbol Table (GEST):* It is used to store each external symbol and its corresponding core address.

External symbol (8 bytes) character	Assigned core (4 bytes) address decimal
"PG1bbbbb"	104
"PG1ENT1b"	124

vi. *Local external symbol array (LESA):* The external symbol is used for relocation and linking purpose. This is used to identify the RLD card by means of an ID number rather than the symbols name. The ID number must match an SD or ER entry on the ESD card

Assigned core address of corresponding symbol [4 bytes]
104
124
134
….
….

This technique saves space and also increases the processing speed.

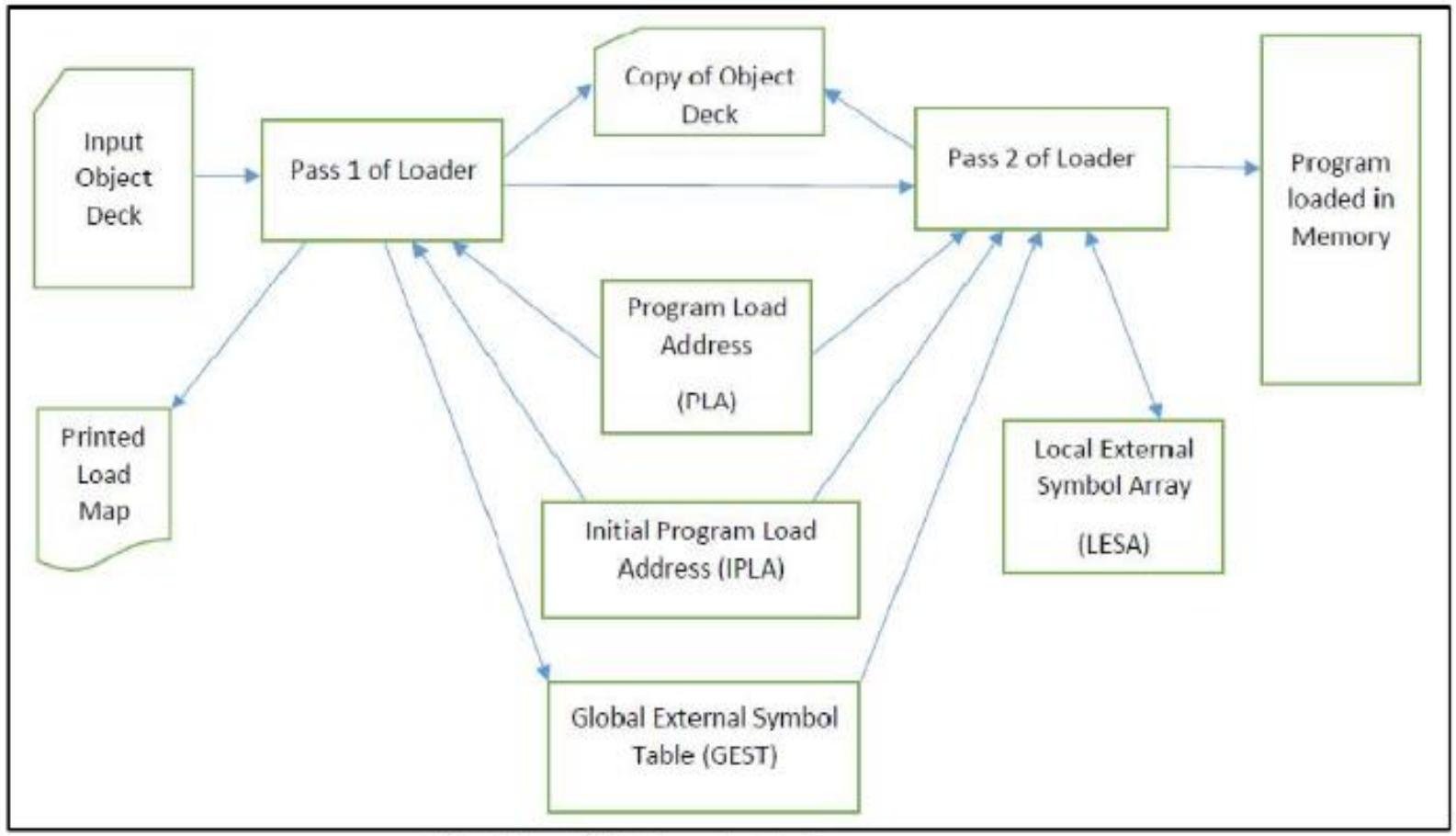


Fig 6: Use of databases by loader passes

4 Algorithm

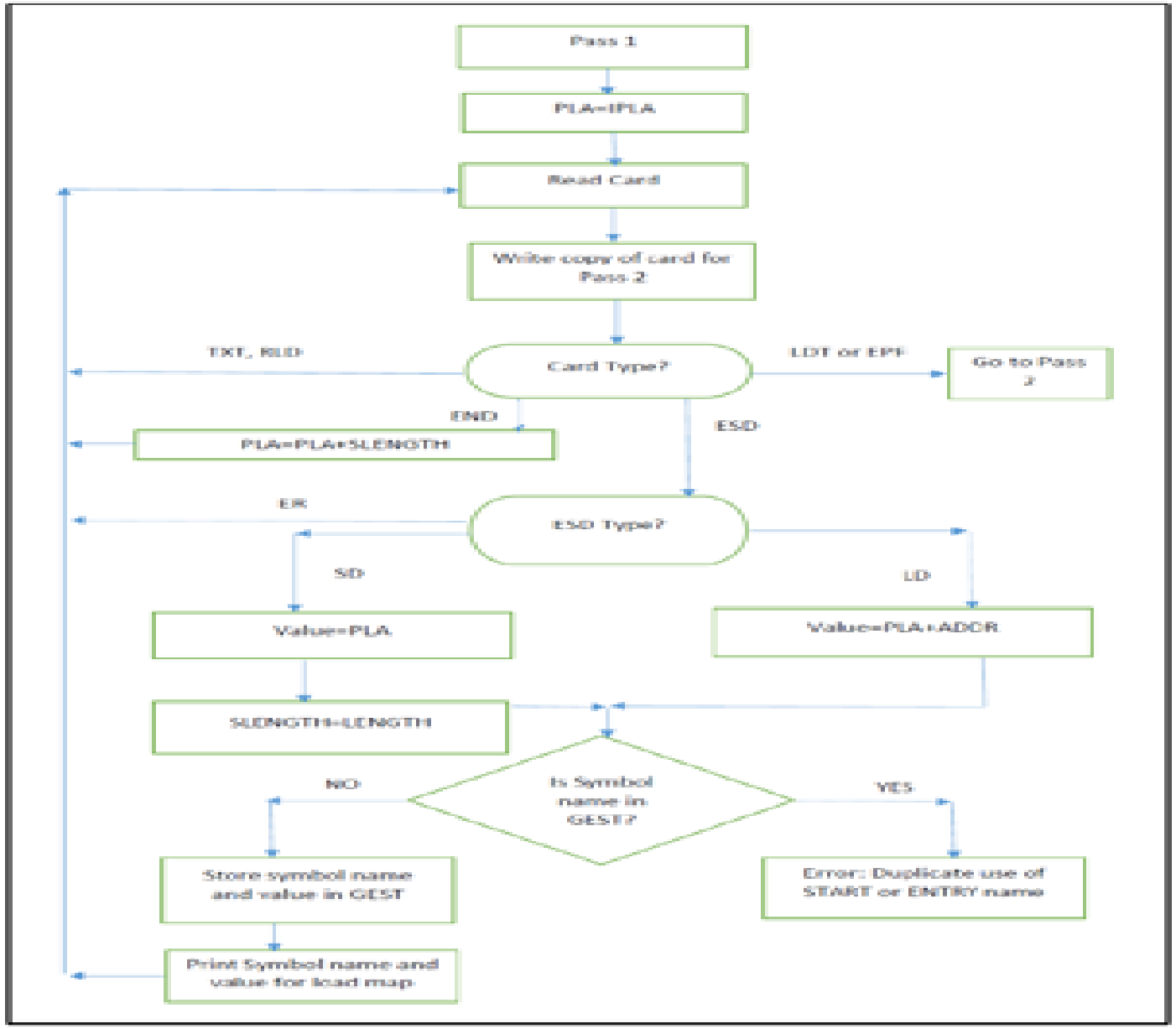
Pass 1: ***Assign location to each segment and also finding the values of all symbols***

Algorithm

1. Initial program load address [PLA] it's set to the initial program load address [IPLA]
2. Read the object card
3. Write copy of card for pass2
4. The card can be any one of the following type
   i) Text card or RLD card- there is no processing required during pass1 and then read the next card
   ii) ESD card is processed based on the type of the external symbols

5. SD card reads the length field LENGTH from the card is temporarily saved in the variable SLENGTH.
   a. The value, VALUE to assign to this symbol is set to the current value of the PLA.
   b. The symbol and its assigned value are then stored in the GEST.
   c. If the symbol already existed in the GEST then this is an error
   d. The symbol and its value are printed as part of the load map.
6. LD is read the value to be assigned is set to the current PLA+ the relative address [ADDR]. The ADDR indicates on the ESD card
7. ER symbols do not required any processing duration pass1

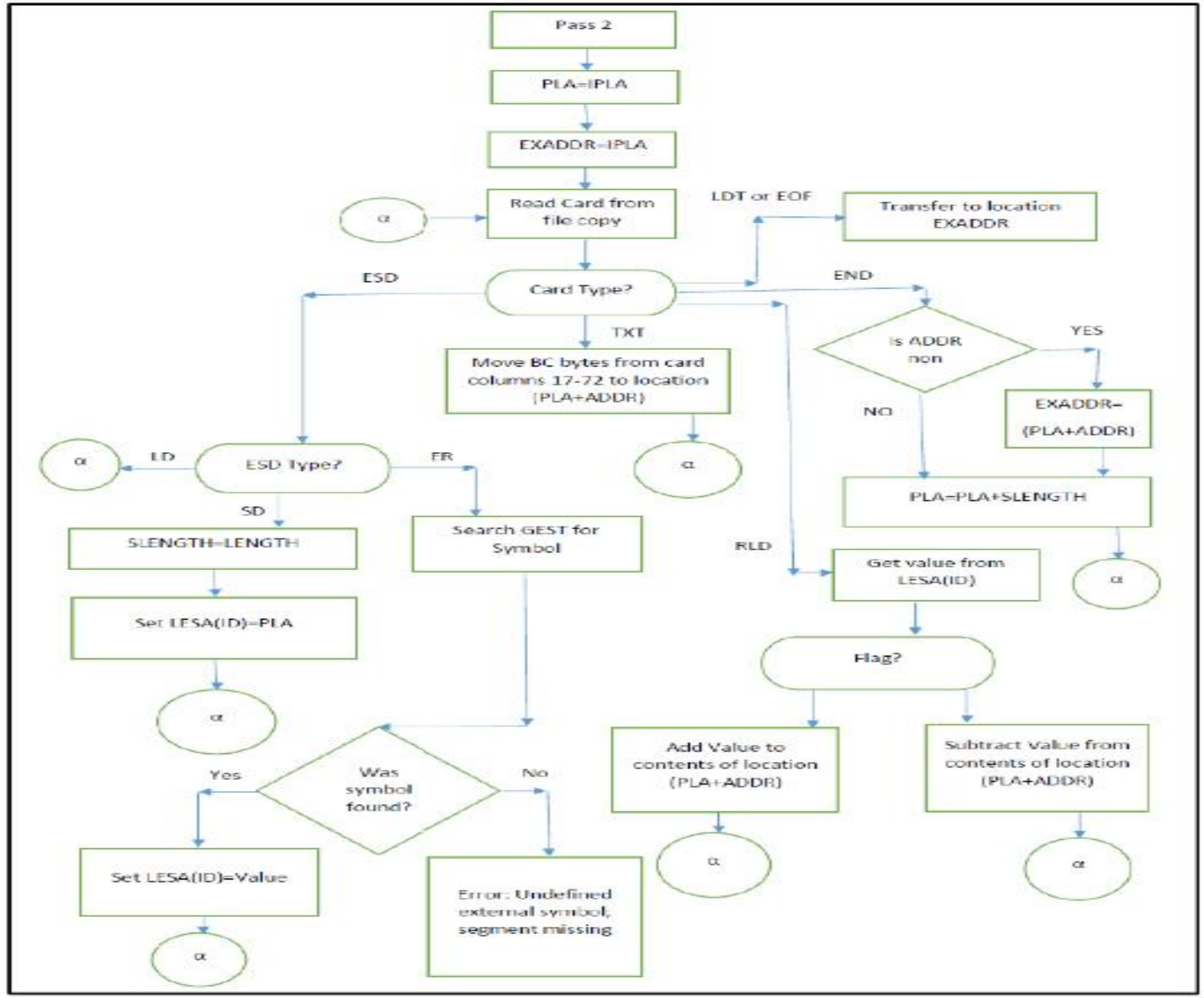
8. When an END card is encountered the program load address is incremented by length of the segment and saved on SLENGTH.
9. EOF card is read pass1 is completed and control transfer to pass2.


Pass 1
PLA=IPLA
Read Card
Write copy of card for Pass 2
TXT, RLD
Card Type?
LDT or EPT
Go to Pass 2
END
PLA=PLA+SLENGTH
ESD
ER
ESD Type?
SD
LD
Value=PLA
Value=PLA+ADDR
SLENGTH=LENGTH
NO
Is Symbol name in GEST?
YES
Store symbol name and value in GEST
Error: Duplicate use of START or ENTRY name
Print Symbol name and value for load map

Pass 2

Algorithm

1. If an address is specified in the END card then that address is used as the executed start address otherwise, the execution will begin from the first segment
2. In pass2 the five cards are read one by one described as follows
1. At the beginning of pass2 the program load address is initialized as in pass1 and the execution start address [EXADDR] is set to IPLA.
    - ***i.*** ***ESD card***
        a) SD type=>the length of the segment is temporarily saved in the variable SLENGTH. The LESA [ID] is set to the current value of the PLA.
        b) LD type=> Does not requires any processing during pass2
        c) ER type=>the guest is searched for a match with the ER symbols. If it is not found then there is an error. If found in the GEST its value is extracted and the corresponding LESA entry is set
    - ii. ***Text card:*** The text card is copied from the card to the appropriate relocated memory location [PLA+ADDR]
    - iii. ***RLD card:*** The value to be used for relocation and linking is executed from the. LESA [ID] as specified by the ID field. Depending upon the flag values is either added to or subroutine from the address constants.
    - iv. ***END card:*** If an execution start address is specified on the END card it is saved in a variable EXADDR. The PLA is incremented by length of the segment and saved in SLENGTH, becoming the PLA for the next segment [PLA=PLA+SLENGTH]
    - v. ***EOF card:*** The loader transfers control to the loaded program at the address specified by the current contents of the execution address variable [EXADDR]


Pass 2
PLA=IPLA
EXADDR=IPLA
α
Read Card from file copy
LDT or EOF
Transfer to location EXADDR
ESD
Card Type?
END
TXT
Move BC bytes from card columns 17-72 to location (PLA+ADDR)
Is ADDR non
YES
NO
EXADDR= (PLA+ADDR)
α
LD
ESD Type?
ER
SD
PLA=PLA+SLENGTH
SLENGTH=LENGTH
Search GEST for Symbol
RLD
Get value from LESA(ID)
α
Set LESA(ID)=PLA
Flag?
α
Yes
Was symbol found?
No
Add Value to contents of location (PLA+ADDR)
Subtract value from contents of location (PLA+ADDR)
Set LESA(ID)=Value
Error: Undefined external symbol, segment missing
α
α
α

Other loading segments:

Binders:

In order to avoid the disadvantages of direct linking divides the loading process into two separate programs:

i. A binder
ii. A module loader

Binder is a program that performs the function as direct linking loader in binding together. It outputs the text as a file or card deck, rather than placing the relocated and linked text directly into memory. The output files are in format ready to be loaded and are called a load module. The module loader loads the module into memory. The binder performs the function of the allocation, relocation and linking

The modules loader performs the function of loading. There are 2 major classes of binders:

i. *Core image builder:* A specific memory allocation of the program is performed at a time that the subroutines are bound together. It is called a core image module and the corresponding binder is called a core image builder

   Advantages: Simple to implement and Fast to execution

   Disadvantages: Difficult to allocate and load the program and Linkage editor

ii. *Linkage editors:* The linkage editor can keep track of relocation information so that the resulting load module can be further relocated and their care the module loader must performs additional allocation and relocation as well as loading but it does not worry about the problem of linking.

   Advantages: More flexible allocation and loading scheme

   Disadvantages: Implementation is so complex

Dynamic Loading:

For the entire loader scheme we have assured that all of the subroutine needed or loaded into memory at the same time.
If the total amount of memory required by all these subroutine exceeds the amount available especially for large programs there is a problem to load all the subroutine into memory at the same time. There are several hardware techniques to solve this problem such as paging or segments. Usually the subroutine of a program is need at different times.

Example:
Pass1 and Pass2 of an assembler are mutually exclusive. The assembler can recognize which subroutine calls the other subroutine it is possible to produce an overlay structure that identifies the mutual exclusive subroutine.

A 25k
B 15k
C 20k
D 30k
E 20k

Diagram Subroutine calls and procedures

Above figure illustrate a program consist of five sub programs [A to E] and that requires 110k bytes of memory. The arrow indicator that sub program "A" only calls „B", „D" and „E" sub program „B" only calls „C" and sub programs „C", „E" does not calls only other sub programs or routines.
Note that procedure „B" and „D" are never use at the same time.

Overlay Loading scheme:

Total required 65K memory is enough at a time. The portion of the loader the necessary procedure is called as overlay supervisor or simply the flipper. This scheme is called as dynamic loading or *load on call (LOCAL).*

Dynamic linking:

The main disadvantages of all of the previous loading scheme are that the sub routine as references but they never executed, but the loader still in use the overhead of linking the subroutines. This mechanism the loading linking of external references are postponed *until execution time* the loaded only the main program if the main program should execute a transfer instruction to an external address or external variable the loader is call.

Advantages

- The number overhead is incurred unless the procedure to be called or reference is actual used.
- System can be dynamically reconfigured
- Saves memory

Disadvantages:

- More complex because of postponed most of binding process until the program execution time.